

PROJECT REPORT

HemoCheck

AI-Powered Anemia Prediction Web Application

A Machine Learning-Based Clinical Screening Tool

Attribute	Detail
Application Name	HemoCheck
Type	Full-Stack Machine Learning Web Application
Frontend	HTML5, CSS3, Vanilla JavaScript
Backend	Python, Flask
Machine Learning	scikit-learn (Random Forest Classifier)
Model Persistence	Pickle (model.pkl)
Report Date	July 2026

1. Introduction

HemoCheck is a full-stack web application that predicts the likelihood of anemia from a patient's Complete Blood Count (CBC) values. It combines a trained machine learning classification model with a clean, responsive, medically-themed user interface, allowing a user to enter their CBC parameters and instantly receive a prediction, a confidence score, a health status, and personalized health guidance.

The goal of the project is to demonstrate an end-to-end ML web application: data generation and model training, a Flask backend that serves predictions through a REST API, and a polished frontend that presents results in an accessible, non-alarming, and informative way.

1.1 Objectives

- Build a responsive, modern, medically-themed web interface for anemia screening.
- Train a supervised classification model capable of distinguishing anemic from non-anemic CBC profiles.
- Serve real-time predictions through a Flask REST API with server-side input validation.
- Present results with a confidence score, health status, and tailored precautions, diet, and doctor-consultation advice.
- Ensure the application is mobile-friendly and visually consistent across devices.

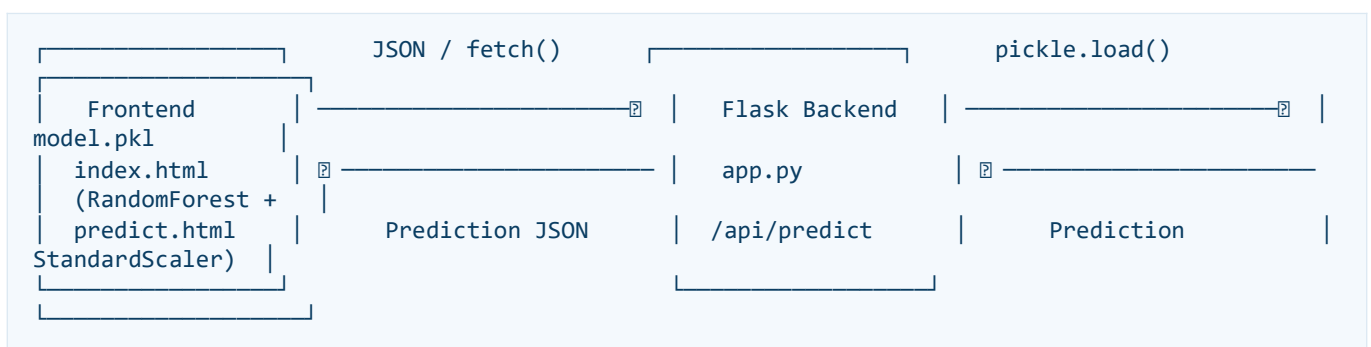
1.2 Scope and Disclaimer

Medical Disclaimer

HemoCheck is an educational and informational screening tool. It is not a certified diagnostic device and does not replace professional medical evaluation. All predictions should be confirmed with a licensed physician and standard laboratory diagnostics.

2. System Architecture

The application follows a simple three-tier architecture: a static frontend (HTML/CSS/JavaScript) communicates with a Flask backend over a JSON REST API, and the backend loads a pre-trained scikit-learn model bundle at startup to serve predictions.



2.1 Project Structure

```

anemia-app/
├── app.py                # Flask backend (routes + prediction API)
├── model.pkl             # Trained model bundle (RandomForest + scaler)
├── requirements.txt      # Python dependencies
├── model/
│   └── train_model.py    # Data generation & model training script
├── templates/
│   ├── index.html        # Home page
│   └── predict.html      # Prediction form + results page
├── static/
│   ├── css/style.css     # Design system, responsive layout, animations
│   └── js/
│       ├── main.js       # Shared behavior (mobile navigation)
│       └── predict.js    # Form validation, API calls, result rendering

```

3. Frontend Design

The interface uses a clinical blue-and-white palette, paired with the Outfit typeface for headings and Inter for body text, to convey a calm, professional, medical tone. The layout is fully responsive, collapsing to a single-column mobile view below 720px, and includes hover states, entrance animations, and a floating animated CBC snapshot card in the hero section.

3.1 Home Page

- Hero section with headline, call-to-action buttons, key statistics, and an animated CBC snapshot card.
- About Anemia section explaining the condition, common symptoms, and normal CBC reference ranges.
- Features section highlighting instant predictions, privacy, confidence scoring, and personalized guidance.
- A four-step "How it Works" process explainer.
- A call-to-action band directing users to the prediction page.
- A footer with navigation, resources, contact information, and a medical disclaimer.

3.2 Prediction Page

The prediction form collects the following ten CBC and demographic parameters, each with inline validation against clinically realistic ranges:

Parameter	Unit	Valid Range
Age	years	0 – 120
Gender	Male / Female	—
Hemoglobin (Hb)	g/dL	3 – 20
RBC Count	M/ μ L	1.0 – 8.0
WBC Count	K/ μ L	1.0 – 30.0
Hematocrit (HCT)	%	10 – 65
Mean Corpuscular Volume (MCV)	fL	50 – 130

Parameter	Unit	Valid Range
Mean Corpuscular Hemoglobin (MCH)	pg	10 – 45
MCH Concentration (MCHC)	g/dL	20 – 40
Platelet Count	$\times 10^3/\mu\text{L}$	10 – 800

Additional UI behavior on the prediction page includes:

- Real-time field-level validation with clear error messages and colored field states.
- A loading spinner on the Predict button while the API call is in progress.
- A Reset button that clears the form and result panel.
- A color-coded result card: teal for "No Anemia", coral for "Anemia Detected".
- An animated confidence bar and toast notifications for input errors.

4. Machine Learning Model

4.1 Dataset

Since no labeled clinical dataset was supplied, a synthetic dataset of 6,000 samples was generated using clinically-grounded CBC reference ranges. Roughly half the samples were simulated as healthy and half as anemic (mild, moderate, and severe), drawn from gender-specific normal distributions with realistic physiological correlations — for example, Hematocrit tracking Hemoglobin via the standard "rule of three", and lower Mean Corpuscular Volume in anemic samples to reflect microcytic presentations.

Note on Production Use

For clinical or production deployment, this synthetic dataset should be replaced with a real, properly licensed and de-identified clinical dataset. The training script (`train_model.py`) is structured so that a real dataset can be substituted directly.

4.2 Model Training Pipeline

1. Generate synthetic CBC data using gender-specific normal distributions for healthy and anemic classes.
2. Split data into training (80%) and testing (20%) sets, stratified by class.
3. Fit a `StandardScaler` on the training features.
4. Train a `RandomForestClassifier` (300 estimators, max depth 10, class-balanced weights).
5. Evaluate on the held-out test set using accuracy and a classification report.
6. Bundle the trained model, scaler, and feature order into a single pickle file (`model.pkl`).

4.3 Model Performance

Metric	No Anemia	Anemia	Overall
Precision	0.99	0.99	—
Recall	0.99	0.99	—

Metric	No Anemia	Anemia	Overall
F1-score	0.99	0.99	—
Accuracy	—	—	99.5%

The high accuracy reflects the clean separability of the synthetic dataset. Real-world CBC data would be expected to show more overlap between classes, and the model should be retrained and re-validated on real clinical data before any diagnostic use.

4.4 Health Status and Recommendations Logic

In addition to the binary ML prediction (Anemia Detected / No Anemia) and its confidence score, the backend applies WHO-based hemoglobin thresholds — adjusted for gender — to classify severity into Normal, Mild, Moderate, or Severe. Each severity level maps to a curated set of precautions, diet recommendations, and doctor-consultation advice, returned alongside the prediction.

5. Backend and API

The Flask backend loads `model.pkl` once at startup and exposes the following routes:

Route	Method	Description
/	GET	Renders the home page
/predict	GET	Renders the prediction form page
/api/predict	POST	Accepts CBC values as JSON, returns prediction, confidence, health status, and recommendations

5.1 Sample Request

```
POST /api/predict
Content-Type: application/json

{
  "Age": 34, "Gender": 0, "Hemoglobin": 7.2, "RBC": 3.0,
  "WBC": 6.5, "HCT": 22, "MCV": 75, "MCH": 22,
  "MCHC": 29, "Platelets": 230
}
```

5.2 Sample Response

```
{
  "success": true,
  "prediction": "Anemia Detected",
  "anemia_detected": true,
```

```

"confidence": 100.0,
"health_status": "Moderate Anemia",
"recommendations": {
  "precautions": ["Reduce physical exertion and rest adequately.", "..."],
  "diet": ["Increase intake of red meat, eggs, beans...", "..."],
  "doctor_advice": "Consult a doctor within the next few days..."
}
}

```

The API also performs server-side validation of every field against the same clinical ranges enforced on the frontend, returning a 400 response with a descriptive error message for missing or out-of-range values.

6. Technology Stack

Layer	Technologies
Frontend	HTML5, CSS3, Vanilla JavaScript, Google Fonts (Outfit, Inter)
Backend	Python 3, Flask
Machine Learning	scikit-learn (RandomForestClassifier, StandardScaler)
Data Handling	Pandas, NumPy
Model Persistence	Pickle

7. Setup and Running the Application

7.1 Prerequisites

- Python 3.9 or later — 64-bit installation strongly recommended (see Section 8).
- pip package manager.

7.2 Installation Steps

```

cd anemia-app
python -m venv venv
venv\Scripts\activate      # Windows
source venv/bin/activate   # macOS / Linux

pip install --upgrade pip
pip install -r requirements.txt

```

7.3 (Re)training the Model

model.pkl is already included and ready to use. To regenerate it after modifying the training script:

```
python model/train_model.py
```

7.4 Running the Server

```
python app.py
```

Then open `http://localhost:5000` in a browser. The home page is served at `/` and the prediction form at `/predict`.

8. Environment Setup: Troubleshooting Log

During deployment on a Windows machine, installation of the pandas dependency failed. This section documents the root cause and resolution for reference.

8.1 Symptom

```
ERROR: Could not find C:\Program Files (x86)\Microsoft Visual Studio
      \Installer\vswhere.exe
```

```
ModuleNotFoundError: No module named 'numpy'
```

8.2 Root Cause

The installed Python interpreter was a 32-bit build (Python 3.9-32). Modern versions of pandas do not ship prebuilt 32-bit Windows wheels, so pip attempted to compile pandas from source using the Meson build system — which in turn requires Microsoft Visual Studio's C++ build tools. Since these were not installed, the build failed, and pandas (along with its dependency, numpy) was never installed.

8.3 Resolution Steps

7. Install a 64-bit Python (3.12) using winget: `winget install -e --id Python.Python.3.12 --source winget`
8. Open a new terminal session so PATH and the "py" launcher pick up the new install.
9. Verify with: `py -0` — confirm a "64-bit" 3.12 entry is listed.
10. Recreate the virtual environment explicitly from the 64-bit interpreter: `py -3.12 -m venv venv`
11. Activate with the correct PowerShell script: `.\venv\Scripts\Activate.ps1` (running `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` first if blocked by policy).
12. Confirm the venv is 64-bit: `python -c "import platform; print(platform.architecture())"` should print `('64bit', ...)`.
13. Install dependencies and run: `pip install -r requirements.txt` then `python app.py`

8.4 Key Lessons

- Creating a virtual environment does not change architecture — a venv inherits the bitness of the interpreter used to create it.
- PowerShell requires the explicit `.\` prefix and `Activate.ps1` filename to activate a venv; the bare `"venv\Scripts\activate"` syntax is a `cmd.exe` convention.
- A newly installed Python version is only visible to `"py -0"` and PATH-based lookups in a freshly opened terminal session.
- Always verify architecture with `platform.architecture()` before installing scientific packages, to avoid compiler-related build failures.

9. Conclusion

HemoCheck successfully demonstrates a complete, production-style machine learning web application: a trained Random Forest classifier served through a Flask REST API, wrapped in a responsive, accessible, medically-themed frontend. The project covers the full lifecycle — synthetic data generation, model training and evaluation, API design with validation, and a polished user experience — while clearly scoping its role as an informational tool rather than a diagnostic device.

Future enhancements could include training on a real, clinically validated dataset, adding user accounts to track CBC history over time, and extending the model to classify anemia subtypes (e.g. iron-deficiency, megaloblastic) based on MCV and MCH patterns.